

Mineral — Firestore Schema & Data Layer Prompt

(v1.4)

Agent-ready specification. Hand this to the build agent in the `mineral-resonance` Firebase project.

Version history at the end of this document.

0. How to read this prompt

This is not a “create these collections” checklist. It is the set of decisions the data layer must encode so you don’t have to invent them. Where a field, rule, or query is specified, treat it as fixed. Where something is explicitly out of scope (§10, §11), do not add it. Mineral is a single-practitioner, single-user-relationship contemplative practice app; privacy of the user’s voice captures is the highest priority, and the pattern engine must be able to quote the user verbatim with attribution while never interpreting on their behalf.

Stack already in place: Expo + React Native + Firebase JS SDK v12, `expo-router`, reused `mineral-resonance` project. Build for iOS, Android, and web.

Produce the eight deliverables in the Deliverables section at the end.

1. Collection structure

Two top-level content collections (global, not user-owned) and one top-level `users` collection whose per-user data lives in **subcollections**. Per-user data is in subcollections — not top-level collections with a `userId` field — so ownership is enforced by path and no `userId`-equality indexes are needed.

<code>encounters/{encounterId}</code>	← global content library (read-all)
<code>practitionerContent/{contentId}</code>	← global, fully locked (admin/console)
<code>users/{uid}</code>	← one doc per user, keyed by Auth UI
<code>users/{uid}/userEncounters/{userEncounterId}</code>	← per-user encounter instances
<code>users/{uid}/fieldNotes/{fieldNoteId}</code>	← per-user captures (most private)
<code>users/{uid}/patterns/{patternType}</code>	← per-user derived data, doc id = pa

users/{uid} — doc id is the Firebase Auth UID

Field	Type	Notes
email	string	
birthDate	timestamp null	
birthTime	string null	e.g. "14:30" ; string preserves "unknown minutes". Optional per product.
birthLocation	{ lat: number, lng: number, label: string } null	
humanDesignType	string null	computed from birth data or null
currentPhase	'signal' 'field' 'friction' 'voice'	Only four values. The Integral / "next turn" is represented by incrementing <code>currentTurn</code> , never as a fifth phase.
currentTurn	number	1-indexed. One turn = one full revolution through the four phases.
journeyStartedAt	timestamp	
createdAt	timestamp	<code>serverTimestamp()</code>
membershipStatus	'free' 'member'	Server-written only (see §3).
membershipSince	timestamp null	Server-written only.
membershipExpiresAt	timestamp null	Server-written only.
membershipProductId	string null	e.g. 'monthly' / 'annual' . Server-written only.
completedEncounterCount	number	Denormalized; drives the "three free, then upgrade" gate. Server-written only , incremented by the completion trigger (§6).

`encounters/{encounterId}` — global content library (you author this)

Field	Type	Notes
<code>title</code>	string	
<code>subtitle</code>	string	
<code>audioPath</code>	string	Cloud Storage path (not a URL); resolve to a download URL at read time.
<code>phase</code>	'signal' 'field' 'friction' 'voice'	
<code>order</code>	number	Sequence within a phase . Drives Today selection (§7).
<code>minTurn</code>	number	Default 1 . Earliest turn this encounter applies to. Lets turn-specific material be added later without migration.
<code>reflectionQuestions</code>	Array<{ id: string, text: string }>	1–3 items, all optional for the user to answer.
<code>integrationPractice</code>	string	
<code>fieldOfferings</code>	Array<{ key: string, text: string }> optional	Inline “from the field” offerings authored <i>with this encounter</i> (motifs likely to appear). Distinct from the keyed library in <code>practitionerContent</code> .

`users/{uid}/userEncounters/{userEncounterId}` — encounter instances

Suggested doc id: ``${encounterId}_t${turn}`` so re-walking on a later turn produces a distinct, idempotent doc.

Field	Type	Notes
<code>encounterId</code>	string	→ <code>encounters/{encounterId}</code>
		Which turn this instance

turn	number	belongs to.
status	'saved' 'in-progress' 'completed'	
startedAt	timestamp null	
completedAt	timestamp null	

No reflection content lives here. Reflections are canonical in `fieldNotes` (§8). This doc holds status + timestamps only.

`users/{uid}/fieldNotes/{fieldNoteId}` — **all captures**

One pool for spontaneous captures **and** encounter reflections, visually distinguished in the UI, never segregated in storage.

Field	Type	Notes
type	'dream' 'spark' 'resistance' 'symbol' 'synchronicity' 'vision' 'desire' 'fear' 'other' 'reflection'	Nine chip types plus <code>reflection</code> .
captureMode	'audio' 'text'	Voice-first; audio is the default.
audioPath	string null	Storage path for audio captures.
content	string null	The final text the engine parses. Typed text for <code>text</code> mode; the transcription result for <code>audio</code> mode (null until transcription completes).
transcriptStatus	'none' 'pending' 'done' 'failed'	'none' for text captures; the audio pipeline moves <code>pending</code> → <code>done/failed</code> .

charge	number null	Engine-derived from content (never user-rated). Scale deliberately unspecified here. Whether charge is intensity/salience ([0, 1]) or signed valence ([-1, 1]) is pinned in the charge-algorithm decision (§11) — the Guide's term "high charge" reads as salience, not valence, so the model must not be baked in by accident. Stored as a single <code>number</code> for now; if the algorithm ends up needing both intensity and valence, revisit the type then (data cost is zero — see below). Null until computed; in v1 always null . Client must not write this (§3).
source	'spontaneous' 'encounter'	
encounterRef	string null	→ a <code>userEncounters</code> doc id, when <code>source === 'encounter'</code> . How reflections attach to their encounter.
questionId	string null	For reflections: which <code>encounters.reflectionQuestions[].id</code> this answers. Powers the revisit view.
atmosphere	'signal' 'field' 'friction' 'voice'	The phase captured in (descriptive, not prescriptive).
createdAt	timestamp	<code>serverTimestamp()</code>

users/{uid}/patterns/{patternType} — derived data (doc id = patternType)

Exactly five docs per user. Doc id is the `patternType` , so reads are doc-id lookups with no composite index.

Field	Type	Notes
patternType	'motif' 'thread' 'resistance' 'condition' 'consciousness'	Equals the doc id.
	Record<string,	item → frequency. Powers the synthesis

itemCounts	number>	sentence and frequency views ("water appears in 6 signals").
exemplars	Record<string, Array<{ text: string, fieldNoteId: string, source: string, capturedAt: timestamp }>>	item → up to 3 representative verbatim quotes with attribution . This is what lets the Guide quote the user exactly ("..." — reflection · the threshold · 3 days ago) from a single read, without touching raw <code>fieldNotes</code> .
offerings	Record<string, { key: string, text: string }>	item → matched <code>practitionerContent</code> offering, copied in by the engine so the user reads counts, exemplars, and offerings from their own doc in one read.
updatedAt	timestamp	

The engine may add pattern-type-specific derived fields (e.g. an average-charge map on the `condition` doc, to support "conditions noted alongside high charge"). Keep any such fields inside the relevant pattern doc; never point back at specific `fieldNotes`.

`practitionerContent/{contentId}` — keyed offering library (fully locked)

Field	Type	Notes
key	string	The motif/resistance/condition this offering is keyed to.
keyType	'motif' 'resistance' 'condition'	Only three of the five pattern types have practitioner-authored offerings. <code>thread</code> (recurring phrases) and <code>consciousness</code> (the structures lens) are user-generated only — no offering library — because their items are too personal (specific phrases the user repeats) or too abstract (Gebser structure distributions) to be keyed in advance.
text	string	"From the field" voice — passive locating constructions, plural, never names the user.

Clients never read this collection directly. The engine matches it against new field notes and copies matches into the user's `patterns.offerings`.

2. Relationships (and non-relationships)

- `fieldNotes` belong to one user (enforced by subcollection path).
- `fieldNotes.encounterRef` (optional) → a `userEncounters` doc id (a reflection belongs to an encounter instance).
- `userEncounters.encounterId` → `encounters/{encounterId}` (an instance of a content encounter).
- `patterns` belong to one user (path), one doc per `patternType`.

Not relationships: field notes do not reference each other (no threading in v1); patterns aggregate, they do not point at specific field notes; encounters are content and do not reference users.

3. Security rules (Firestore)

Ownership is path-based. Spell these out exactly:

- `users/{uid}` — read if `request.auth.uid == uid`. create / update if `request.auth.uid == uid` **AND the write does not change any server-controlled field:** `membershipStatus`, `membershipSince`, `membershipExpiresAt`, `membershipProductId`, `completedEncounterCount`. On update, require each of those equals its existing value (`request.resource.data.X == resource.data.X`); on create, require **all five** at their defaults: `membershipStatus == 'free'`, `completedEncounterCount == 0`, and `membershipSince`, `membershipExpiresAt`, `membershipProductId` each `== null` — otherwise a client could create its doc with a forged `membershipExpiresAt` and pass the guard. These fields are written only by the Admin SDK / billing webhook. Without this guard a user can self-upgrade to `member`.
- `users/{uid}/userEncounters/{id}` — read, write if `request.auth.uid == uid`.
- `users/{uid}/fieldNotes/{id}` — read, write if `request.auth.uid == uid`, **except** the client may not set or modify `charge` (engine-only; require it absent on create and unchanged on update). **Precision note:** Firestore distinguishes a field set to `null` from a field that is absent. The rule requires `charge absent` (`!('charge' in request.resource.data)` on create), so `createFieldNote` must **omit the field entirely** — never write `charge: null`. Only the engine (Admin SDK) ever writes this field. Do **not** attempt to rate-limit creates inside the security rule: rules can `get()` a specific document but cannot query or aggregate, so there is no rule-expressible way to count creates in a time window. Abuse from non-genuine clients is handled by **App Check** (§3a), not by rules. If you want protection against an accidental client-side runaway

loop, add a lightweight throttle on the capture action in the UI — but treat it as a UX guard, not a security control, since any client can bypass it.

- **users/{uid}/patterns/{patternType}** — read: `if request.auth.uid == uid`; write: `if false`. (The engine writes via the Admin SDK, which bypasses rules — “Cloud Functions write” is expressed as `if false` for clients, not as a rule that detects functions.)
- **encounters/{id}** — read: `if request.auth != null`; write: `if false` (authored via console / Admin SDK).
- **practitionerContent/{id}** — read, write: `if false` (fully locked; console / Admin SDK only).

3a. App Check

Enable App Check and enforce it on Firestore and Cloud Storage. This is the actual control against requests from anything other than the genuine app binary — scripted clients, scraped configs, replayed tokens — which is the threat the (removed) rule-based rate limit was reaching for and could not deliver. Attestation providers: **App Attest / DeviceCheck** on iOS, **Play Integrity** on Android, **reCAPTCHA v3** on web (free, sufficient for most apps; upgrade to reCAPTCHA Enterprise only if you have evidence the heuristics aren’t enough — Enterprise is paid and adds quotas + analytics but is overkill for v1).

Wire the provider into the Firebase init alongside Auth/Firestore/Storage. **Staged rollout mechanism, specifically:** ship App Check wired up but enforcement *disabled* in the Firebase console. Production logs will show attestation results for every request without rejecting any. Watch for several days. Once $\geq 99\%$ of legitimate requests are attesting successfully, flip enforcement on in the console — this is a single toggle, not a code change, so a misconfigured build is recoverable by toggling enforcement back off. App Check is per-request and orthogonal to the per-user ownership rules above; both apply.

4. Storage structure & rules

The most private data — voice recordings — lives in Storage, so it needs rules in parallel with Firestore.

```
encounters/{encounterId}/...      ← encounter audio (public content)
users/{uid}/fieldNotes/{noteId}/... ← user voice captures (private)
```

- **encounters/{encounterId}/{file}** — read: `if request.auth != null`; write: `if`

`false` (uploaded via console / Admin SDK).

- `users/{uid}/fieldNotes/{noteId}/{file}` — read, write: if `request.auth.uid == uid`.
- Everything else: deny.

Consider capping audio size and constraining content type on write (e.g.

```
request.resource.size < 25 * 1024 * 1024 &&  
request.resource.contentType.matches('audio/.*')
```

Audio retention policy. Audio files persist indefinitely after transcription. The transcript stored in `content` is the index; the audio file at `audioPath` is the truth. Users can play back their original recordings from past encounter views and field note details — the felt experience of hearing oneself think six months later is part of the practice. Storage cost is acceptable through several thousand users; note it is cumulative *across kept notes*, since each captured-and-kept audio adds to the running total. A long-tenured user’s stored audio grows over time rather than holding at a flat monthly figure. Do not implement automatic expiry.

Delete cascades to audio. “No automatic expiry” is not “never delete.” When a user deletes a field note, the delete must remove **both** the Firestore document **and** its Storage object at `audioPath` — otherwise the recording outlives the note the user believes they erased, which is a privacy failure. Indefinite retention applies only to notes the user keeps; a user-initiated delete is permanent across both stores. **Ordering matters because the two deletes cannot be atomic** (no transaction spans Firestore + Storage): delete the **Storage object first, then the Firestore doc**. If the Storage delete fails, abort and surface a retry — the note remains visible and the user can delete again. The reverse failure mode (doc deleted, audio orphaned) is the one to prevent, because an orphaned recording has nothing pointing at it and will never be cleaned up. The `deleteFieldNote` helper (deliverable #5) owns this sequenced cascade.

5. Indexes (`firestore.indexes.json`)

Because per-user data is path-scoped and `patterns` is a doc-id lookup, most of the original index set disappears. Define, with `queryScope: COLLECTION`:

1. **encounters** — `phase ASC, order ASC` — the Today selection query (§7).
2. **userEncounters** — `status ASC, completedAt DESC` — the “what you’ve walked through” list (completed, most recent first).

Not needed now (single-field `orderBy` is auto-indexed): the chronological field-notes feed (`createdAt DESC` only). Add `fieldNotes type ASC, createdAt DESC` only when a type-filtered feed ships. Do not pre-declare it now.

6. Data flow (pattern engine)

Raw data lives in `fieldNotes` ; derived data lives in `patterns` . The Guide reads only `patterns` . The pipeline:

On `fieldNotes` create:

1. If `captureMode === 'audio'` and `transcriptStatus === 'pending'` : transcribe `audioPath` , write the result to `content` , set `transcriptStatus = 'done'` (or `'failed'`). For `text` captures (`transcriptStatus === 'none'`), skip straight to step 2.
2. Derive `charge` from `content` (sentiment-based; counts/derives, never diagnoses).
3. Parse `content` for motifs / recurring phrases / signal categories. For each relevant `patternType` , update the user's pattern doc: increment `itemCounts` , upsert `exemplars` (verbatim text + `fieldNoteId` + `source` + `capturedAt` , capped at 3 per item), and match against `practitionerContent` to copy any offering into `offerings` . Set `updatedAt` .

On `userEncounters` update to `status === 'completed'` :

- Increment `users/{uid}.completedEncounterCount` .
- Evaluate phase/turn advancement (pacing is by depth of engagement, not days elapsed) and update `currentPhase` / `currentTurn` if warranted.

The engine runs server-side with the Admin SDK, which is why client writes to `patterns` , and to the protected `users` and `fieldNotes` fields, are denied.

Engine implementation is out of scope for this prompt. Define the data shapes the engine consumes (`fieldNotes` writes) and produces (`patterns` updates, `users` field increments), and ensure the security rules deny client writes to engine-controlled fields. Do not implement the Cloud Functions themselves — that work is specified in a follow-up prompt (see §11). The schema must support the data flow described above; the runtime that executes the flow comes next.

What this means for the v1 build. Because transcription and the engine are deferred (§11), this build ships with capture working but the derived layer inert: new audio notes are created at `transcriptStatus: 'pending'` with `content: null` , no `patterns` documents

are written, and the Guide therefore renders empty. **This is the intended milestone, not a defect** — a blank Guide and not-yet-transcribed audio notes are the expected state until the engine prompt ships. `pending` audio notes form the backlog that the transcription/engine pipeline drains once it exists. So “set `transcriptStatus` correctly” in deliverable #5 means: new audio captures are written `pending`, new text captures `none`; nothing in this build moves a note past `pending`.

7. Today's encounter selection

“Today” is the next uncompleted encounter in the user's current phase for the current turn:

1. Read `currentPhase` and `currentTurn` from the user doc.
2. Query `encounters` where `phase == currentPhase` `orderBy order asc` (uses index #1).
3. Filter `minTurn <= currentTurn` **client-side** (do not put this in the query — a `minTurn` range combined with the `phase` equality and `order` sort hits Firestore's range/orderBy restriction; the encounter library is small, so client-side filtering is fine).
4. Read the user's `userEncounters` for the current turn and build the set of completed `encounterId`s (doc ids follow ``_${encounterId}_t${currentTurn}``).
5. Today = the first encounter in the ordered, turn-filtered list whose id is not in the completed set.

Second-turn behavior (open product question, resolved at the schema level): because `userEncounters` are turn-scoped, re-walking the same encounters on turn 2 naturally creates fresh instance docs and the same selection logic works unchanged. If you later author turn-specific material, give it a `minTurn` and an `order`; it slots into the sequence with no migration. The content decision (same encounters deeper vs. new material) can be deferred without touching the schema.

8. Reflections → field notes

A reflection captured during an encounter is a `fieldNote` with `type: 'reflection'`, `source: 'encounter'`, `encounterRef` set to the `userEncounters` doc id, and `questionId` set to the question it answers. `userEncounters` holds only status/timestamps. The encounter revisit view (“when you listened to The Mirror, you noted...”) is a `fieldNotes` query filtered by `encounterRef`. One source of truth, no duplicated reflection text.

9. Auth persistence fix (`firebase.ts`)

The current `firebase.ts` uses `inMemoryPersistence` on native, which logs users out on every cold start — fatal for a daily-return practice. The “v12 removed `getReactNativePersistence`” note is a misdiagnosis: the function exists at runtime; what’s flaky is its TypeScript export. Restore persistence using `AsyncStorage` (already installed:

`@react-native-async-storage/async-storage`):

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import { getApp, getApps, initializeApp } from "firebase/app";
import {
  Auth,
  browserLocalPersistence,
  getAuth,
  initializeAuth,
  // @ts-ignore — present at runtime in the RN bundle; missing from some v12 type
  getReactNativePersistence,
} from "firebase/auth";
import { Platform } from "react-native";

const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApp();

function initAuth(): Auth {
  try {
    if (Platform.OS === "web") {
      return initializeAuth(app, { persistence: browserLocalPersistence });
    }
    return initializeAuth(app, {
      persistence: getReactNativePersistence(AsyncStorage),
    });
  } catch {
    // Already initialized (hot reload).
    return getAuth(app);
  }
}
```

Verify against the exact `firebase@12.13` build; if the import path complains, `firebase/auth/react-native` is the fallback. The goal is simply that sessions survive cold start on native.

10. Out of scope (do not build)

- No social features (no following, no sharing).
- No analytics-events collection (use Firebase Analytics).
- No multi-tenant / organization structure.
- No versioning of encounters or field notes.
- No soft-delete (delete is delete, for v1). Reflections (field notes with `source: 'encounter'`) follow the same delete semantics as spontaneous captures — deletion is permanent. Revisited encounter views will show only currently-existing reflections; a previously-completed encounter whose reflections have been deleted will show its `userEncounters` doc as `completed` but with no attached reflections. This is by design — users get the consequences of their deletions.
- Deletes cascade to Storage. Deleting any audio-mode field note also removes its Storage object at `audioPath` — see §4 and the `deleteFieldNote` helper in deliverables.
- No threading or related-notes between field notes.
- No client writes to `patterns` , `encounters` , or `practitionerContent` .
- Membership entitlement is server-side (billing webhook / Function), never client-asserted.

11. Out of scope for this prompt — deferred to follow-up prompts

The following systems are referenced in this document but their implementation is deferred:

- **The pattern engine itself.** §6 describes the data flow (fieldNote create → transcribe → derive charge → parse motifs → update patterns). The Cloud Functions implementing this flow are out of scope here. For this prompt, ensure the schema and rules support the flow; do not implement the engine.
- **Transcription service selection and integration.** §6 step 1 says "transcribe `audioPath` " without specifying with what. Options include OpenAI Whisper API, Google Speech-to-Text, or on-device transcription. The decision involves trade-offs between accuracy, cost, latency, and data residency, and should be made in a separate decision document. For this prompt, the `transcriptStatus` field machinery should exist (`'none'` | `'pending'` | `'done'` | `'failed'`) but the actual transcription pipeline is not built.
- **charge computation algorithm, including its scale.** §1 specifies the field type

(`number | null`) but deliberately leaves the *scale* open — the choice between intensity (`[0, 1]`) and signed valence (`[-1, 1]`) belongs with the algorithm, because the Guide’s “high charge” reads as salience and must not be silently committed to a valence model. That decision, plus the algorithm itself (sentiment/salience approach, model or API selection, prompt design, fallback behavior on failure), is a separate document. In `v1`, `charge` is always `null`.

- **Membership entitlement enforcement.** §1 lists the membership fields as server-written-only and §3 specifies the security rule guards. The actual billing integration (RevenueCat, Stripe, or platform IAP webhooks → Cloud Function → user doc update) is deferred. For this prompt, ensure the fields exist and the rules deny client writes.

Where this prompt’s deliverables would otherwise stub the above systems, leave the relevant fields as `null` or in their initial state rather than implementing stubs. A field at `null` is a clear signal “not yet computed”; a stub returning fake values is a future bug.

12. Deploying from Replit

The deliverables are files in the repo; **none of them are live until deployed**. Rules in particular: an undeployed `firestore.rules` with the project still in test mode means the database is wide open regardless of what the file says. Deploy is part of this work, not a follow-up.

a. firebase.json wiring. The deploy CLI needs a `firebase.json` at the repo root declaring where the rules and indexes live:

```
{
  "firestore": {
    "rules": "firestore.rules",
    "indexes": "firestore.indexes.json"
  },
  "storage": {
    "rules": "storage.rules"
  }
}
```

Also create `.firebaserc` pinning the project so deploys can’t hit the wrong one:

```
{ "projects": { "default": "mineral-resonance" } }
```

b. CLI install. Add `firebase-tools` as a dev dependency (`npm i -D firebase-tools`) and

invoke it via `npx firebase ...` — do not rely on a global install that won't survive Replit environment rebuilds.

c. Authentication from the Replit shell. Two options:

- *Interactive (fine for a solo founder):* `npx firebase login --no-localhost` — prints a URL to open in your own browser, you authorize, paste the resulting code back into the shell. Sessions persist in the workspace until revoked.
- *Non-interactive (for repeatable/CI deploys):* create a service account in Google Cloud Console with least privilege (**Firestore Rules Admin** + **Cloud Datastore Index Admin** is sufficient for rules + indexes; do not use the default Editor role), download its JSON key, store the JSON **in Replit Secrets** (never commit it), write it to a temp file at shell start, and point `GOOGLE_APPLICATION_CREDENTIALS` at that file.

d. Test before first deploy. At minimum, exercise the rules in the **Rules Playground** (Firestore console → Firestore → Rules) against the critical cases: another user reading your fieldNotes (deny), a client writing `membershipStatus: 'member'` (deny), a client creating a fieldNote with `charge` present (deny), the owner reading their own data (allow). Better: run the emulator (`npx firebase emulators:start --only firestore,storage`) with `@firebase/rules-unit-testing` and encode those cases as tests. The four cases above are the ones that, if wrong, are silent until they're a breach.

e. Deploy command.

```
npx firebase deploy --only firestore:rules,firestore:indexes,storage --project mi
```

f. Verify. In the console, the Rules tab shows a publish timestamp — confirm it's the deploy you just ran. Note that **index builds are asynchronous**: the deploy returns immediately but composite indexes take minutes to build, and queries needing them error with a console link until ready. This is normal; don't re-deploy in response.

13. Firebase project hardening checklist

Project-level security that lives outside the rules files. Work through all of it; each item is small.

- **API keys are identifiers, not secrets.** The Firebase web API key in the client config does not grant data access — rules + App Check are the security boundary. It will ship inside the app bundle and that is fine. **But still restrict each key** in Google Cloud Console → Credentials: the browser key by HTTP referrer (your real domains only), the Android key by package name + SHA-1, the iOS key by bundle ID. Firebase creates

separate keys per registered app — restrict each by its own platform, don't share one unrestricted key across platforms.

- **Client env vars are not secret.** `EXPO_PUBLIC_*` variables are embedded in the shipped bundle — acceptable for the `firebaseConfig` values, **never** for service-account JSON, admin credentials, or any API key that grants privileged access. Anything privileged lives in Replit Secrets and is used only server-side / at deploy time.
- **Production mode from day one.** Firestore and Storage must never run in “test mode” (allow-all-for-30-days). If the project was created in test mode, deploying the §3/§4 rules replaces that — which is another reason deploy is part of this work, not a follow-up.
- **Auth provider lockdown.** Enable **Email/Password only**; leave every other provider off until there's a product reason. Confirm **email enumeration protection** is on (default for newer projects; verify under Authentication → Settings). Trim **authorized domains** to your real domains plus localhost for development.
- **Billing alert.** Set a budget alert in Google Cloud Billing (e.g. \$25/month with notifications at 50/90/100%). A runaway read loop or scripted abuse shows up first as cost; the alert is your tripwire.
- **IAM minimalism.** The project owner is you; the deploy service account (§12c) has the two narrow roles named above; nothing else has access. Resist granting Editor “to make things work.”
- **Point-in-time recovery (when real users arrive).** Users' voice reflections are irreplaceable. Before opening the beta beyond yourself, enable Firestore PITR (paid, 7-day window) or schedule regular exports to a Storage bucket. Not needed while the only data is your own testing.
- **App Check** — covered in §3a; it belongs on this checklist too. Wired in this build, enforcement flipped on after the observation window.

Deliverables

Produce all eight:

1. `firestore.rules` — implementing §3 exactly, including the server-controlled-field guards on `users` and the `charge` guard on `fieldNotes`.
2. `storage.rules` — implementing §4.

3. `firestore.indexes.json` — the two indexes in §5, `queryScope: COLLECTION`.
 4. **TypeScript types** (single file, e.g. `types/firestore.ts`) — interfaces for every collection plus the shared enums (`PhaseId` , `FieldNoteType` , `PatternType` , `MembershipStatus` , `EncounterStatus`).
 5. **Thin data-access helpers** — at minimum: the Today selection query (§7), the chronological field-notes feed, the “walked through” list, a `createFieldNote` that sets `captureMode` / `transcriptStatus` correctly (audio → `pending` , text → `none`) and **omits charge entirely** (never writes `charge: null` — the rule requires the field absent), and a `deleteFieldNote` that performs the sequenced cascade: Storage object first, then the Firestore document, aborting with a surfaced retry if the Storage delete fails (§4).
 6. **Corrected** `firebase.ts` — the §9 auth persistence fix.
 7. **App Check setup** (§3a) — provider wiring in the Firebase init (App Attest/DeviceCheck, Play Integrity, reCAPTCHA v3) and enforcement enabled on Firestore and Storage, rolled out in stages.
 8. **Deploy wiring** (§12) — `firebase.json` and `.firebaserc` at the repo root, `firebase-tools` as a dev dependency, and the rules/indexes/storage deploy executed and verified (Rules tab publish timestamp + the four Rules Playground denial/allow cases checked).
-

Changelog

v1.4 (current). Final validation pass. Three precision fixes: (1) `charge` null-vs-absent resolved — clients must omit the field entirely, rule checks `!('charge' in request.resource.data)` on create, `createFieldNote` never writes `charge: null` ; (2) create-time guard on `users/{uid}` extended to require all five server-controlled fields at defaults, including the three membership timestamps/`productId == null` , closing the forged- `membershipExpiresAt` -on-create hole; (3) `deleteFieldNote` cascade ordering specified — Storage object first, then Firestore doc, abort + retry on Storage failure, since the two deletes cannot be atomic and an orphaned recording is the unrecoverable failure mode. Two new sections: §12 Deploying from Replit (`firebase.json`/.`firebaserc` wiring, CLI auth options, Rules Playground test cases, deploy command, async index-build note) and §13 Firebase project hardening checklist (per-platform API key restrictions, client env vars vs. Replit Secrets, production mode, auth lockdown, billing alert, IAM minimalism, PITR before beta). Deliverable #8 added for the deploy wiring.

v1.3. Polish pass on v1.2. Changelog moved to end of document (was stacking at top). reCAPTCHA guidance specified (v3 free + sufficient; Enterprise only when needed). App

Check staged-rollout mechanism made concrete (enforcement-disabled-then-flip via console toggle). Audio retention paragraph clarified to specify cumulative cost applies across *kept* notes only. §10 reflection-deletion bullet split for readability.

v1.2. Rule-based rate limiting removed (not expressible in Firestore rules) and replaced with App Check (§3a). Audio delete-cascade reconciled with the retention policy (§4, §10). v1 build end-state stated explicitly (§6). `charge` scale deferred rather than pinned, since the Guide's "high charge" reads as salience, not valence (§1, §11). App Check and a `deleteFieldNote` helper added to deliverables.

v1.1. Pattern engine / transcription / charge computation / billing entitlement explicitly scoped out and deferred to follow-up prompts (see §11). `charge` field spec tightened. `practitionerContent` keying clarified. Audio retention policy made explicit in §4. Reflection delete semantics clarified in §10.

v1. Initial schema specification.